Amazon CloudFront — Console Dashboard Wise Important Interview Questions & Answers

CloudFront Console Structure

CloudFront
│
├── Dashboard
│
├── Distributions
│ └── Individual Distribution
│ ├── General
│ ├── Origins
│ ├── Behaviors
│ ├── Error Pages
│ ├── Invalidations
│ ├── Monitoring
│ └── Security
│
├── Origin Access
│ ├── Origin Access Control (OAC)
│ └── Origin Access Identity (OAI)
│
├── Policies
│ ├── Cache Policies
│ ├── Origin Request Policies
│ └── Response Headers Policies
│
├── Functions
│ ├── CloudFront Functions
│ └── Key Value Stores
│
├── Public Keys
├── Key Groups
├── Continuous Deployment
├── Distribution Tenants
└── Settings

---

1. What is Amazon CloudFront?

Answer:

Amazon CloudFront is AWS's Content Delivery Network (CDN) service.
It delivers website content from locations closer to users, which reduces latency and improves performance.

CloudFront can deliver static and dynamic content using HTTP or HTTPS. **A**  AWS Documentation

For example:

Suppose our website is hosted on an S3 bucket in Mumbai.
A user in the US requests an image, and CloudFront serves it from a nearby edge location if cached.
This reduces the need to fetch the image from Mumbai every time.

Diagram:

User
↓
CloudFront Edge Location
↓
S3 / ALB / EC2
↓
Application Content

Simple ga gurthupettuko:

CloudFront = CDN + Faster content delivery

CloudFront Dashboard

2. What do we see in the CloudFront Dashboard?

Answer:

The CloudFront Dashboard gives an overall view of CloudFront distributions and their status.
It helps us monitor distributions, traffic, errors, and important CloudFront activity.
From here, we can move to distributions and manage individual CloudFront configurations.

For example:

Suppose our company has three CloudFront distributions.
From the dashboard, we can identify the distributions and check their current state before troubleshooting.

Diagram:

CloudFront Dashboard

↓
Distributions
↓
Status / Traffic / Monitoring
Simple ga gurthupettuko:
Dashboard = Overall CloudFront view

Distributions
3. What is a CloudFront Distribution?
Answer:
A CloudFront distribution is the main configuration that tells CloudFront how to deliver content to users.
It defines the origin, cache behaviors, security settings, domains, and other delivery settings.

CloudFront uses the distribution configuration to serve content through its edge network. **A** AWS Documentation
For example:
Suppose our application files are stored in S3.
We create a CloudFront distribution and configure the S3 bucket as the origin.
Users then access the content through the CloudFront distribution.
Diagram:
User
↓
CloudFront Distribution
↓
Origin
↓
S3
Simple ga gurthupettuko:
Distribution = Complete CloudFront delivery configuration

---

4. What is a CloudFront Distribution ID?
Answer:
A Distribution ID is a unique identifier assigned to a CloudFront distribution.
It is used to identify that particular distribution when working with AWS APIs and management operations.

The Distribution ID cannot be changed after the distribution is created. **A** AWS Documentation
For example:
Suppose we have a distribution with ID E123ABC456.
When we need to update or manage that distribution through AWS tools, this ID identifies the correct distribution.
Diagram:
CloudFront
↓
Distribution
↓
Distribution ID
↓
E123ABC456
Simple ga gurthupettuko:
Distribution ID = Unique ID of CloudFront distribution

Individual Distribution — General
5. What is the General section of a CloudFront Distribution?
Answer:
The General section contains important information and basic configuration of a distribution.
It can include the distribution domain name, status, distribution ID, alternate domain names, and related settings.
It is useful when checking the overall configuration of a distribution.
For example:
Suppose our application uses www.myapp.com.
We can check the distribution's domain, status, and alternate domain configuration from the distribution details.
Diagram:
Distribution
↓
General
├— Domain
├— Status
├— Distribution ID
└— Domains

Simple ga gurthupettuko:
General = Basic distribution information

---

6. What is the CloudFront Distribution Domain Name?
Answer:
CloudFront provides a default domain name for every distribution.
It normally looks similar to d111111abcdef8.cloudfront.net.

Users can access content through this domain, or we can configure our own domain using an alternate domain name. **A** AWS Documentation
For example:
Suppose CloudFront gives our distribution d123example.cloudfront.net.
We can use that address for testing before connecting our own domain such as www.myapp.com.
Diagram:
CloudFront
↓
Distribution
↓
d123example.cloudfront.net
↓
Content
Simple ga gurthupettuko:
CloudFront Domain = AWS-provided distribution URL

Origins
7. What is an Origin in CloudFront?
Answer:
An origin is the location where the original content is stored.
CloudFront gets content from the origin when it is not already available in the cache.

An origin can be an S3 bucket, load balancer, HTTP server, or other supported source. **A** AWS Documentation+1
For example:
Suppose our website files are stored in an S3 bucket.
We configure that S3 bucket as the CloudFront origin.
When CloudFront needs the original file, it gets it from S3.
Diagram:
User
↓
CloudFront
↓
Origin
↓
S3 Bucket
Simple ga gurthupettuko:
Origin = Where original content lives

---

8. What types of Origins can CloudFront use?
Answer:
CloudFront can use services such as S3, Elastic Load Balancing, HTTP servers, and other supported AWS origins.
The origin depends on where the application or content is running.

A distribution can also have multiple origins for different types of content. **A** AWS Documentation
For example:
We can keep images in S3 and dynamic application traffic behind an ALB.
CloudFront can use both as origins and route requests using cache behaviors.
Diagram:
CloudFront
/ \
↓ ↓
S3 ALB
Static Dynamic
Content Content
Simple ga gurthupettuko:
Origin = Source of CloudFront content

---

9. What is Origin Domain?
Answer:
Origin Domain identifies the DNS name of the resource from which CloudFront gets content.
It can represent an S3 bucket, load balancer, HTTP server, or another supported origin.

CloudFront uses this value to know where to send requests when content is needed. **A**  AWS Documentation
For example:
Suppose our S3 bucket contains website files.
We configure the bucket as the origin domain.
CloudFront then knows where to fetch the original content.
Diagram:
CloudFront
↓
Origin Domain
↓
S3 / ALB / HTTP Server
Simple ga gurthupettuko:
Origin Domain = Address of content source

---

10. What is Origin Path?
Answer:
Origin Path allows CloudFront to add a specific path when requesting content from an origin.
It is useful when the required files are stored inside a particular directory or path.

This avoids needing to include that path repeatedly in viewer requests. **A**  AWS Documentation
For example:
Suppose the origin contains content under /production.
We can configure /production as the origin path so CloudFront requests content from that location.
Diagram:
CloudFront
↓
Origin
↓
/production
↓
index.html
Simple ga gurthupettuko:
Origin Path = Path inside the origin

Origin Access Control
11. What is Origin Access Control (OAC)?
Answer:
Origin Access Control allows CloudFront to securely access an S3 origin.
It helps keep the S3 bucket private while allowing CloudFront to retrieve the required objects.

The S3 bucket policy can be configured to allow access from the CloudFront distribution. **A**  AWS Documentation
For example:
Suppose our S3 bucket contains private website files.
Instead of making the bucket public, we configure CloudFront with OAC.
Users access the files through CloudFront while the S3 bucket remains private.
Diagram:
User
↓
CloudFront
↓
OAC
↓
Private S3 Bucket
Simple ga gurthupettuko:
OAC = Secure CloudFront → Private S3 access

---

12. What is the difference between OAC and making S3 public?
Answer:
With OAC, the S3 bucket can remain private and CloudFront is allowed to access the required objects.

With a public bucket, users may access S3 objects directly if the permissions allow it.
OAC is generally preferred for a secure CloudFront-to-S3 architecture.
For example:
We have a private S3 bucket containing application files.
We use OAC so users access files through CloudFront instead of directly accessing the S3 bucket.
This gives us better control over how content is delivered.
Diagram:
Public S3:
User → S3

Private S3 + OAC:
User → CloudFront → OAC → S3
Simple ga gurthupettuko:
OAC = Keep S3 private + CloudFront access

Behaviors
13. What is a Cache Behavior in CloudFront?
Answer:
A cache behavior defines how CloudFront handles requests that match a particular path pattern.
It can control the origin, HTTPS policy, allowed HTTP methods, caching, query strings, cookies, headers, and other settings.


The default behavior normally handles requests that do not match another behavior. **A** AWS Documentation
For example:
Suppose /images/* contains static images and /api/* contains API requests.
We can create different cache behaviors so images are cached heavily while API requests use different rules.
Diagram:
CloudFront
├── /images/* → S3 → Cache
└── /api/* → ALB → Application
Simple ga gurthupettuko:
Cache Behavior = Rule for handling requests

---

14. What is a Path Pattern in CloudFront?
Answer:
A path pattern tells CloudFront which requests should use a particular cache behavior.
Patterns can represent paths such as /images/*, /api/*, or other URL structures.


CloudFront compares viewer requests with the configured path patterns to select the appropriate behavior. **A** AWS Documentation
For example:
If a user requests /images/logo.jpg, it can match /images/*.
CloudFront then applies the cache behavior configured for image requests.
Diagram:
Request:
/images/logo.jpg
↓
Path Pattern:
/images/*
↓
Cache Behavior
Simple ga gurthupettuko:
Path Pattern = Decides which behavior applies

---

15. What is the Default Cache Behavior?
Answer:
The default cache behavior is the main behavior automatically created for a CloudFront distribution.
It normally handles requests that do not match a more specific cache behavior.


It defines important settings such as the origin, viewer protocol policy, methods, and caching behavior. **A** AWS Documentation
For example:
Suppose we have only one website origin.
The default behavior can handle requests such as /index.html, /about.html, and /logo.png.
Diagram:
User Request

↓
CloudFront
↓
No Specific Match
↓
Default Behavior
↓
Origin
Simple ga gurthupettuko:
Default Behavior = Main request-handling rule

Viewer Protocol Policy
16. What is Viewer Protocol Policy?
Answer:
Viewer Protocol Policy controls whether users can access CloudFront using HTTP or HTTPS.
Common options include HTTP and HTTPS, Redirect HTTP to HTTPS, and HTTPS Only.

For secure websites, Redirect HTTP to HTTPS or HTTPS Only is commonly used. **A**  AWS Documentation
For example:
Suppose a user opens http://www.myapp.com.
If we configure Redirect HTTP to HTTPS, CloudFront redirects the user to the HTTPS version.
Diagram:
User
↓
HTTP Request
↓
CloudFront
↓
Redirect
↓
HTTPS
Simple ga gurthupettuko:
Viewer Protocol = HTTP/HTTPS rule

HTTP Methods
17. What are Allowed HTTP Methods in CloudFront?
Answer:
Allowed HTTP Methods define which HTTP methods CloudFront accepts and can forward to the origin.
Common methods include GET, HEAD, POST, PUT, PATCH, DELETE, and OPTIONS depending on the configuration.

For static content, GET and HEAD are commonly enough. **A**  AWS Documentation
For example:
For a website serving images and HTML, we may allow only GET and HEAD.
For an API that receives POST requests, we may need additional methods.
Diagram:
User
↓
CloudFront
↓
GET / POST / PUT
↓
Allowed?
↓
Origin
Simple ga gurthupettuko:
Allowed Methods = Which HTTP requests CloudFront accepts

Cache Policies
18. What is a Cache Policy?
Answer:
A Cache Policy controls what values CloudFront uses when creating the cache key.
It can consider URL query strings, HTTP headers, and cookies when deciding whether two requests should use the same cached object.

Using a proper cache policy can improve the cache hit ratio and reduce origin requests. **A**  AWS Documentation

For example:
Suppose /product?id=10 and /product?id=20 return different products.
The cache policy can include the id query parameter so CloudFront keeps the correct cached version for each product.
Diagram:
Request
↓
Headers + Cookies + Query Strings
↓
Cache Policy
↓
Cache Key
↓
Cached Object
Simple ga gurthupettuko:
Cache Policy = Decides what makes cached content different

---

19. What is an Origin Request Policy?
Answer:
An Origin Request Policy controls which headers, cookies, and query strings CloudFront sends to the origin.
It is separate from the cache key decision.
This is useful when the origin needs request information even when that information should not create separate cache entries.
For example:
Suppose our application needs a specific HTTP header to process a request.
We can configure the origin request policy to forward that header to the origin.
Diagram:
User Request
↓
CloudFront
↓
Origin Request Policy
↓
Required Headers / Cookies / Query Strings
↓
Origin
Simple ga gurthupettuko:
Origin Request Policy = What CloudFront sends to origin

TTL / Caching
20. What is TTL in CloudFront?
Answer:
TTL stands for Time To Live and controls how long an object can remain in the CloudFront cache.
CloudFront uses minimum, default, and maximum TTL settings depending on the cache configuration.

Proper TTL values can reduce origin requests and improve application performance. **A**  AWS Documentation
For example:
Suppose our logo rarely changes.
We can keep it cached for a longer period so users do not repeatedly request it from the origin.
Diagram:
Origin
↓
CloudFront Cache
↓
TTL = Cache Duration
↓
User Requests
Simple ga gurthupettuko:
TTL = How long content stays cached

Cache Hit / Miss
21. What is a CloudFront Cache Hit?
Answer:
A cache hit happens when CloudFront already has the requested object in its cache.
CloudFront can return the object directly without requesting it from the origin.
This improves response time and reduces load on the origin.
For example:
A user requests logo.png and CloudFront already has it cached.
CloudFront immediately returns the cached image.
The S3 bucket does not need to send the file again.
Diagram:

User
↓
CloudFront Cache
↓
Object Found
↓
Cache Hit
↓
User
Simple ga gurthupettuko:
Cache Hit = Object already available in CloudFront

---

22. What is a Cache Miss?
Answer:
A cache miss happens when CloudFront does not have the requested object in its cache.
CloudFront then requests the object from the configured origin.
After receiving it, CloudFront can cache it according to the configured caching rules.
For example:
A user requests a new image that CloudFront has never cached.
CloudFront requests the image from S3.
It returns the image to the user and can cache it for future requests.
Diagram:
User
↓
CloudFront Cache
↓
Object Not Found
↓
Cache Miss
↓
S3 / Origin
↓
CloudFront
↓
User
Simple ga gurthupettuko:
Cache Miss = CloudFront must contact origin

Invalidations
23. What is a CloudFront Invalidation?
Answer:
An invalidation tells CloudFront to remove specific cached objects from edge caches.
It is useful when content has changed and we want CloudFront to fetch the latest version from the origin.
We can invalidate specific paths or broader paths depending on the requirement.
For example:
Suppose we update index.html in S3 but users are still receiving the old cached file.
We can create an invalidation for /index.html.
CloudFront then fetches the updated file when needed.
Diagram:
S3
↓
Updated File
↓
CloudFront Cache
↓
Invalidation
↓
Old Cache Removed
↓
New File
Simple ga gurthupettuko:
Invalidation = Remove old cached content

Custom Error Pages
24. What are CloudFront Custom Error Pages?
Answer:
Custom error pages allow us to configure what CloudFront should return when errors such as 403 or 404 occur.
We can configure a custom response page and control how the error is handled.
This helps provide a better user experience instead of showing a generic error.

For example:
Suppose a user requests a page that does not exist.
Instead of showing a basic 404 error, CloudFront can return our custom error.html page.
Diagram:
User
↓
CloudFront
↓
404 Error
↓
Custom Error Page
↓
User
Simple ga gurthupettuko:
Custom Error = User-friendly error response

Alternate Domain Names
25. What is an Alternate Domain Name in CloudFront?
Answer:
An Alternate Domain Name, also called CNAME, allows us to use our own domain name with a CloudFront distribution.
Instead of using only the AWS-generated CloudFront domain, users can access the application through a custom domain.
A valid TLS certificate is normally configured when HTTPS is required.
For example:
CloudFront provides d123example.cloudfront.net.
We can configure www.myapp.com as an alternate domain and use Route 53 to point the domain to CloudFront.
Diagram:
www.myapp.com
↓
Route 53
↓
CloudFront
↓
Origin
Simple ga gurthupettuko:
Alternate Domain = Use your own domain with CloudFront

SSL / TLS
26. Why do we configure an SSL/TLS certificate in CloudFront?
Answer:
An SSL/TLS certificate allows CloudFront to serve content securely over HTTPS.
It encrypts communication between the viewer and CloudFront.
For a custom CloudFront domain, we configure an appropriate certificate for that domain.
For example:
Suppose users access www.myapp.com.
We configure a certificate for the domain so users can securely access the website using HTTPS.
Diagram:
User
↓
HTTPS
↓
CloudFront
↓
TLS Certificate
↓
Secure Connection
Simple ga gurthupettuko:
Certificate = HTTPS security for CloudFront

Geographic Restrictions
27. What are CloudFront Geographic Restrictions?
Answer:
Geographic restrictions allow us to control access to CloudFront content based on the geographic location of viewers.
We can create restrictions using an allowlist or blocklist of countries.
This is useful when business or legal requirements require country-level access control.
For example:
Suppose our application should only be available to users in India and Singapore.
We can configure geographic restrictions so requests from other countries are blocked.
Diagram:
Users
↓
CloudFront

↓
Geographic Restriction
├— India → Allow
├— Singapore → Allow
└— Other → Block
Simple ga gurthupettuko:
Geographic Restriction = Country-based access control

Signed URLs / Signed Cookies
28. What are Signed URLs in CloudFront?
Answer:
A signed URL provides controlled access to private CloudFront content.
It contains information that CloudFront uses to verify whether the request is authorized and valid.
It is useful when specific users should access protected content for a limited period.
For example:
Suppose we have a private video in S3.
Instead of making it public, our application generates a signed URL for an authorized user.
The user can access the video using that URL.
Diagram:
User
↓
Application
↓
Signed URL
↓
CloudFront
↓
Private Content
Simple ga gurthupettuko:
Signed URL = Temporary controlled content access

Response Headers Policy
29. What is a Response Headers Policy?
Answer:
A Response Headers Policy controls HTTP headers that CloudFront adds to responses sent to viewers.
It can be used for security headers, CORS headers, and other response-header requirements.
This helps standardize response headers without changing the origin application.
For example:
Suppose our website needs security-related HTTP response headers.
We can configure a response headers policy in CloudFront and attach it to the required cache behavior.
Diagram:
Origin
↓
CloudFront
↓
Response Headers Policy
↓
Security / CORS Headers
↓
User
Simple ga gurthupettuko:
Response Headers Policy = Control headers sent to users

CloudFront Functions
30. What are CloudFront Functions?
Answer:
CloudFront Functions are lightweight functions that run at CloudFront edge locations.
They are useful for simple request and response transformations with very low latency.
Common use cases include URL redirects, URL rewrites, and modifying HTTP headers.
For example:
Suppose users request /old-page but the application now uses /new-page.
A CloudFront Function can redirect or rewrite the request at the edge before sending it to the origin.
Diagram:
User
↓
CloudFront Edge
↓
CloudFront Function
↓
Rewrite / Redirect
↓

Origin
Simple ga gurthupettuko:
CloudFront Function = Small logic at edge

Lambda@Edge
31. What is Lambda@Edge?
Answer:
Lambda@Edge allows Lambda functions to run closer to CloudFront viewers and customize request or response processing.
It is useful for more advanced processing than simple CloudFront Functions.
It can be used with different CloudFront request and response events.
For example:
Suppose we need advanced request processing before a request reaches the application.
We can use Lambda@Edge to execute custom Lambda logic as part of the CloudFront request flow.
Diagram:
User
↓
CloudFront Edge
↓
Lambda@Edge
↓
Processing
↓
Origin
Simple ga gurthupettuko:
Lambda@Edge = Advanced logic at CloudFront edge

Public Keys & Key Groups
32. What is a CloudFront Public Key?
Answer:
A CloudFront public key is used as part of the mechanism for securing private content with signed URLs or signed cookies.
CloudFront uses the configured key information to validate signatures.
It helps CloudFront verify that a request for protected content is properly authorized.
For example:
Suppose our application provides private premium videos.
We use signing information so only authorized requests can access the content through CloudFront.
Diagram:
Application
↓
Signed Request
↓
CloudFront
↓
Public Key Verification
↓
Private Content
Simple ga gurthupettuko:
Public Key = Verify signed CloudFront requests

---

33. What is a CloudFront Key Group?
Answer:
A Key Group is a collection of public keys used with CloudFront signed URLs or signed cookies.
It provides a manageable way to associate trusted public keys with a CloudFront distribution.
This is useful when multiple signing keys need to be managed.
For example:
Suppose our company has multiple public keys for authorized content access.
We can place those keys into a key group and configure CloudFront to trust that group.
Diagram:
Public Key 1 ─┐
Public Key 2 ─┼→ Key Group → CloudFront
Public Key 3 ─┘
Simple ga gurthupettuko:
Key Group = Collection of trusted public keys

Monitoring
34. How do we monitor CloudFront?
Answer:
CloudFront provides monitoring information about requests, traffic, errors, and cache behavior.
CloudWatch can be used for metrics and alarms related to CloudFront activity.
Monitoring helps us identify performance problems, high error rates, and unusual traffic.
For example:
Suppose users report that the website is slow.

We can check CloudFront metrics and cache behavior to understand whether requests are reaching the origin frequently or errors are increasing.

Diagram:

Users
↓
CloudFront
↓
Metrics
↓
CloudWatch
↓
Dashboard / Alarms

Simple ga gurthupettuko:

CloudFront Monitoring = Traffic + Errors + Performance

Continuous Deployment

35. What is CloudFront Continuous Deployment?

Answer:

CloudFront Continuous Deployment helps us safely test configuration changes before applying them to the primary distribution.
It allows a staging configuration to receive selected traffic for testing.
This reduces the risk of directly changing the production CloudFront configuration.

For example:

Suppose we want to test a new cache behavior.
We can test the configuration with a staging setup first and verify that the application works correctly before moving the change to production.

Diagram:

Production Distribution
↓
Continuous Deployment
↓
Staging Configuration
↓
Test Traffic

Simple ga gurthupettuko:

Continuous Deployment = Safely test CloudFront changes

Distribution Tenants

36. What are CloudFront Distribution Tenants?

Answer:

CloudFront distribution tenants are used with the multi-tenant CloudFront architecture.
A multi-tenant distribution can provide common settings that can be inherited by multiple distribution tenants.



Each tenant can represent an individual website or domain while reusing shared configuration. **A** AWS Documentation

For example:

Suppose a SaaS company has hundreds of customer websites.
Instead of maintaining completely separate configurations, it can use a multi-tenant distribution with individual tenants.

Diagram:

Multi-Tenant Distribution
|
┌──┼──┐
↓ ↓ ↓
Tenant A Tenant B Tenant C
↓ ↓ ↓
Site A Site B Site C

Simple ga gurthupettuko:

Distribution Tenant = Individual site using shared CloudFront setup

---

Important Architecture Questions

37. How does CloudFront work with S3?

Answer:

CloudFront can use an S3 bucket as an origin and distribute its objects through edge locations.
When content is cached, CloudFront can serve it without contacting S3 for every request.



For private S3 content, Origin Access Control can be used to allow CloudFront access securely. **A** AWS Documentation

For example:

Suppose our website files are stored in S3.

A user requests logo.png, and CloudFront checks its cache first.
If the file is not cached, CloudFront gets it from S3 and returns it to the user.
Diagram:
User
↓
CloudFront
↓
Cache
↓
S3 Origin
Simple ga gurthupettuko:
CloudFront + S3 = Fast and secure static content delivery

---

38. How does CloudFront work with an Application Load Balancer?
Answer:
CloudFront can use an Application Load Balancer as an origin.
The user sends a request to CloudFront, and CloudFront can forward the request to the ALB when the content is not served from cache or should go to the origin.
The ALB then distributes application traffic to backend targets.
For example:
Suppose our application runs on EC2 instances behind an ALB.
CloudFront sits in front of the ALB and handles global content delivery and caching.
Dynamic requests can be forwarded from CloudFront to the ALB.
Diagram:
User
↓
CloudFront
↓
ALB
↓
EC2
↓
Application
Simple ga gurthupettuko:
CloudFront = Global delivery, ALB = Application traffic distribution

---

39. Why do we use CloudFront with Route 53?
Answer:
Route 53 manages DNS while CloudFront provides content delivery through the edge network.
Route 53 can point our custom domain to the CloudFront distribution.
This gives users a simple domain name while CloudFront handles content delivery.
For example:
Suppose our website is www.myapp.com.
Route 53 sends the domain to CloudFront, and CloudFront serves the website from the nearest suitable edge location.
Diagram:
User
↓
www.myapp.com
↓
Route 53
↓
CloudFront
↓
Origin
Simple ga gurthupettuko:
Route 53 = DNS, CloudFront = CDN

---

40. What is the difference between CloudFront and S3?
Answer:
S3 is mainly used to store objects and files, while CloudFront is used to deliver content efficiently to users.
S3 can act as an origin for CloudFront.
CloudFront caches content at edge locations, while S3 stores the original objects.
For example:
We store index.html and images in S3.
CloudFront takes those files from S3 and delivers them closer to users.
So S3 stores the content and CloudFront delivers it.
Diagram:
S3
↓
Stores Content
↓

CloudFront
↓
Delivers Content
↓
Users
Simple ga gurthupettuko:
S3 = Store, CloudFront = Deliver

---

41. What happens when a user requests content from CloudFront?
Answer:
The user first sends the request to the CloudFront distribution.
CloudFront checks the appropriate cache behavior and then checks whether the requested object is cached.
If it is a cache hit, CloudFront returns it; otherwise, CloudFront gets it from the origin and can cache it.
For example:
A user requests image.jpg.
If CloudFront already has the image, it returns it immediately.
If not, CloudFront gets it from S3 and then serves the user.
Diagram:
User
↓
CloudFront
↓
Cache?
├— Yes → Return Object
└— No → Origin → Cache → User
Simple ga gurthupettuko:
CloudFront = Check cache → If needed go to origin

---

42. What is the main difference between CloudFront Functions and Lambda@Edge?
Answer:
CloudFront Functions are designed for lightweight, fast edge processing.
Lambda@Edge is intended for more advanced request and response processing.
We choose between them based on the complexity and requirements of the edge logic.
For example:
For a simple URL redirect, CloudFront Functions can be suitable.
For more advanced request processing, Lambda@Edge may be appropriate.
Diagram:
CloudFront Edge
│
├— CloudFront Function
│ ↓
│ Simple Logic
│
└— Lambda@Edge
↓
Advanced Logic
Simple ga gurthupettuko:
Function = Simple/Fast, Lambda@Edge = Advanced
43. What are the most important CloudFront console areas for an interview?
Answer:
The most important areas are Distributions, Origins, Behaviors, Cache Policies, Origin Request Policies, and Origin Access Control.
We should also understand invalidations, HTTPS, alternate domain names, error pages, monitoring, and geographic restrictions.
For advanced topics, CloudFront Functions, Lambda@Edge, signed URLs, key groups, and continuous deployment are important.
For example:
If an interviewer asks how you secure and optimize a CloudFront distribution, we should explain OAC, HTTPS, cache policies, TTL, behaviors, and monitoring.
These concepts together explain most real-world CloudFront setups.
Diagram:
CloudFront
│
├— Distribution
├— Origins
├— Behaviors
├— Cache Policies
├— OAC
├— HTTPS
├— Invalidations
├— Monitoring
└— Edge Functions **Simple ga gurthupettuko:**
CloudFront = Distribution + Origin + Behavior + Cache + Security + Monitoring